

Habitat UTS – Documentación técnica del proyecto

Parcial de Programación Java (UTS) — Aplicación web de inmobiliaria en JSP/JDBC/MariaDB

Bryan Rojas

1. Visión general
 - 1.1 Tecnologías
 - 1.2 Estructura de carpetas
2. Arquitectura y flujo de la aplicación
 - 2.1 Diagrama del flujo de autenticación
 - 2.2 Diagrama del control de acceso a rutas privadas
3. Seguridad
 - 3.1 Filtro de control de acceso — `FiltroControlAcceso.java`
 - 3.2 Segunda capa — `seguridad.jspf`
 - 3.3 Contraseñas y consultas seguras
4. Fragmentos JSP compartidos (`WEB-INF/jspf/`)
 - 4.1 `conexion.jspf` — acceso a datos
 - 4.2 `utilidades.jspf` — funciones comunes
 - 4.3 `seguridad.jspf` — autorización por rol (ver §3.2)
 - 4.4 `cabecera.jspf` — layout superior
 - 4.5 `pie.jspf` — layout inferior
5. Páginas públicas
6. Módulo Cliente (`/cliente/*`)
7. Módulo Inmobiliaria (`/inmobiliaria/*`)
8. Módulo Administrador (`/administrador/*`)
9. Base de datos — `inmobiliaria_uts`
 - 9.1 Tablas y su propósito
 - 9.2 Reglas de integridad destacadas
 - 9.3 Diagrama entidad-relación (simplificado)
 - 9.4 Extracto del script (DDL de las tablas centrales)
 - 9.5 Datos de prueba y usuarios
10. Configuración y despliegue
 - 10.1 Conexión a la base de datos
 - 10.2 Pasos de instalación
 - 10.3 `web.xml` — configuración de la aplicación
11. Resumen de cobertura funcional

1. Visión general

Habitat UTS es una aplicación web de inmobiliaria construida con **JSP puro** (sin Servlets de negocio ni DAO), **JDBC** directo y **MariaDB** como motor de base de datos, siguiendo el mismo estilo que el proyecto de referencia *RestauranteWeb*: fragmentos `.jspf` compartidos, consultas SQL escritas directamente en cada JSP y Bootstrap 5 para la interfaz.

Permite a un visitante consultar el catálogo público de propiedades, y a un usuario autenticado — según su rol — guardar favoritos, agendar visitas, radicar solicitudes de compra/arriendo, gestionar propiedades, atender citas y solicitudes, o administrar usuarios, catálogos y auditoría.

1.1 Tecnologías

- **Java 8+ / Jakarta-Servlet API 4.0** (paquete `javax.servlet`, Tomcat 9)
- **JSP** y fragmentos JSP (`.jsp`) incluidos con `<% include %>`
- **JDBC** con el driver `mariadb-java-client-3.5.10.jar`
- **MariaDB** (base de datos `inmobiliaria_uts`)
- **Bootstrap 5.3.3** y **Bootstrap Icons 1.11.3** (vía CDN)
- **HTML, CSS** propio (`estilos.css`) y **JavaScript** del navegador (el de Bootstrap)

1.2 Estructura de carpetas

```
parcialEntrega/
├── sql/
│   └── 01_modelo_inmobiliaria_mariadb.sql # script de creación + datos de prueba
├── src/com/uts/inmobiliaria/
│   └── FiltroControlAcceso.java # único Servlet: Filter de seguridad
└── web/ # raíz desplegable en Tomcat
```

```

├── WEB-INF/
│   ├── classes/           # .class compilado del filtro
│   ├── jspf/             # fragmentos JSP compartidos
│   ├── lib/mariadb-java-client-3.5.10.jar
│   └── web.xml            # registro del filtro, bienvenida, encoding
├── administrador/        (módulo ADMINISTRADOR)
├── cliente/              (módulo CLIENTE)
├── inmobiliaria/        (módulo INMOBILIARIA)
├── css/estilos.css
├── *.jsp                 (páginas públicas: index, login, registro, catálogo, detalle...)
└── README.md

```

Esta separación por carpeta de rol (`cliente/` , `inmobiliaria/` , `administrador/`) es la base del control de acceso: el `Filter` intercepta cualquier petición a esas rutas y decide si continúa o redirige.

2. Arquitectura y flujo de la aplicación

1. El visitante entra por `index.jsp` y puede navegar el catálogo público (`propiedades.jsp` , `detalle.jsp`) sin autenticarse.
2. Para guardar favoritos, agendar citas, radicar solicitudes o administrar, debe **registrarse** (`registro.jsp` → `registro_guardar.jsp`) e **iniciar sesión** (`login.jsp` → `acceso.jsp`).
3. `acceso.jsp` valida correo + contraseña cifrada (SHA-256) y el estado `activo` de la cuenta contra la tabla `usuario`.
4. Si la validación es correcta, se guardan en la **sesión HTTP** el `idUsuario` , `nombre` y `rol` del usuario.
5. A partir de ahí, dos capas de seguridad protegen las páginas privadas:
 - o **FiltroControlAcceso** (Servlet `Filter` , declarado en `web.xml`), que intercepta `/cliente/*` , `/inmobiliaria/*` y `/administrador/*`.
 - o **seguridad.jspf** , incluido al inicio de cada JSP privado, que además valida que el rol de sesión esté en la lista `rolesPermitidos` propia de esa página.
6. Cada JSP privado o público que necesita datos abre su propia conexión JDBC (vía `conexion.jspf`), ejecuta sentencias `PreparedStatement` contra MariaDB y cierra los recursos en un bloque `finally`.
7. El layout (cabecera, menú según rol, pie) se arma con los fragmentos `cabecera.jspf` y `pie.jspf` , y utilidades comunes (cifrado, formato de pesos, parsing seguro, escape HTML, color de estado) viven en `utilidades.jspf`.

2.1 Diagrama del flujo de autenticación

```

[Login] --POST--> acceso.jsp --SELECT usuario+rol--> MariaDB
      |
      | credenciales inválidas / cuenta inactiva
      | --> redirige a login.jsp?error=...
      |
      | credenciales válidas
      | --> session.idUsuario / nombre / rol
      | --> redirige a /<rol>/panel.jsp

```

2.2 Diagrama del control de acceso a rutas privadas

```

Petición a /cliente/* , /inmobiliaria/* , /administrador/*
      |
      | v
      | FiltroControlAcceso (Servlet Filter, web.xml)
      | |-- sin sesión -----> redirige a login.jsp?error=sesion
      | |-- rol no coincide con ruta --> redirige a acceso-denegado.jsp
      | |-- rol coincide -----> chain.doFilter() -> continúa al JSP
      |
      | v
      | seguridad.jspf (segunda validación
      |                 contra el arreglo rolesPermitidos
      |                 definido en cada página)

```

3. Seguridad

3.1 Filtro de control de acceso — `FiltroControlAcceso.java`

Es el único Servlet del proyecto (todo lo demás son JSP). Se registra en `web.xml` con tres `filter-mapping` sobre `/cliente/*` , `/inmobiliaria/*` y `/administrador/*`.

Lógica de `doFilter` :

1. Obtiene la sesión sin crear una nueva (`getSession(false)`).
2. Calcula la ruta relativa al contexto (`ruta = URI - contextPath`).
3. Si no hay sesión o no hay `idUsuario` guardado, redirige a `login.jsp?error=sesion`.
4. Si hay sesión, compara el prefijo de la ruta (`/cliente/` , `/inmobiliaria/` , `/administrador/`) contra el `rol` guardado en sesión.

5. Si el rol no corresponde a la ruta, redirige a `acceso-denegado.jsp`.
6. Si todo es correcto, continúa la cadena con `chain.doFilter(request, response)`.

```
public class FiltroControlAcceso implements Filter {
    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
        FilterChain chain) throws IOException, ServletException {
        HttpServletRequest httpRequest = (HttpServletRequest) request;
        HttpServletResponse httpResponse = (HttpServletResponse) response;
        HttpSession session = httpRequest.getSession(false);
        String contexto = httpRequest.getContextPath();
        String ruta = httpRequest.getRequestURI().substring(contexto.length());

        if (session == null || session.getAttribute("idUsuario") == null) {
            httpResponse.sendRedirect(contexto + "/login.jsp?error=sesion");
            return;
        }
        String rol = (String) session.getAttribute("rol");
        boolean permitido = (ruta.startsWith("/cliente/") && "CLIENTE".equals(rol))
            || (ruta.startsWith("/inmobiliaria/") && "INMOBILIARIA".equals(rol))
            || (ruta.startsWith("/administrador/") && "ADMINISTRADOR".equals(rol));
        if (!permitido) {
            httpResponse.sendRedirect(contexto + "/acceso-denegado.jsp");
            return;
        }
        chain.doFilter(request, response);
    }
}
```

3.2 Segunda capa — seguridad.jspf

Cada JSP privado declara antes de incluir este fragmento un arreglo `String[] rolesPermitidos` (por ejemplo `{"CLIENTE"}` o `{"INMOBILIARIA"}`). El fragmento:

1. Lee `idUsuario`, `rol` y nombre de la sesión.
2. Si no hay sesión, redirige a `login.jsp?error=sesion`.
3. Recorre `rolesPermitidos` buscando coincidencia con el rol de sesión; si no la encuentra, redirige a `acceso-denegado.jsp`.

Esta doble validación (Filter + fragmento) es intencional: el `Filter` protege el árbol completo de carpetas, y el fragmento afina el permiso página por página (por ejemplo, dentro de `/administrador/*` distintas páginas podrían en el futuro exigir sub-roles distintos).

3.3 Contraseñas y consultas seguras

- Las contraseñas nunca se guardan en texto plano: se cifran con **SHA-256** sobre la combinación `correo:contraseña` (función `claveCifrada` en `utilidades.jspf`), tanto al insertar el usuario como al validar el login. El mismo esquema de hash se replica en el script SQL con `SHA2(correo:clave, 256)` para los usuarios de prueba.
- **Todas** las consultas usan `PreparedStatement` con parámetros ligados (`?`), nunca concatenación de texto, evitando inyección SQL.
- La salida de cualquier dato proveniente del usuario o de la base de datos se escapa con la función `esc()` (reemplaza `&`, `<`, `>`, `"`, `'`) antes de imprimirse en HTML, mitigando XSS reflejado/almacenado.
- Las operaciones que escriben más de una tabla (crear cuenta + asignar rol, crear solicitud + documento, actualizar propiedad + imagen + características) usan `con.setAutoCommit(false)` y `con.commit()` / `deshacer(con)` (`rollback`) para mantener la integridad transaccional.

4. Fragmentos JSP compartidos (WEB-INF/jspf/)

Como en el proyecto no se usan Servlets de negocio ni DAO, toda la lógica reutilizable vive en cinco fragmentos que cada JSP incluye con `<%@ include file="/WEB-INF/jspf/..." %>`. Al ser un `include` de traducción (no un `<jsp:include>` en tiempo de ejecución), el código se copia textualmente dentro del JSP que lo incluye, por lo que las variables declaradas en un fragmento deben tener nombres que no choquen con las del JSP anfitrión.

4.1 conexion.jspf — acceso a datos

Declara las constantes de conexión y tres métodos de utilidad JDBC:

- `abrirConexion()`: carga el driver `org.mariadb.jdbc.Driver` y devuelve una `Connection` a `jdbc:mariadb://localhost:3306/inmobiliaria_uts` con el usuario configurado.
- `cerrar(AutoCloseable... recursos)`: cierra en orden inverso cualquier combinación de `ResultSet`, `PreparedStatement` y `Connection`, ignorando errores de cierre.
- `deshacer(Connection con)`: hace `rollback()` de forma segura cuando una transacción falla.

```
public Connection abrirConexion() throws SQLException {
    try { Class.forName(DB_DRIVER); }
    catch (ClassNotFoundException ex) {
        throw new SQLException("No se encontro el driver de MariaDB. Agregue el JAR a WEB-INF/lib.", ex);
    }
    return DriverManager.getConnection(DB_URL, DB_USUARIO, DB_CLAVE);
}
```

4.2 utilidades.jspf — funciones comunes

Método	Propósito
sha256(texto)	Calcula el resumen SHA-256 en hexadecimal de un texto.
claveCifrada(usuario, clave)	Aplica sha256(usuario + ":" + clave) ; es el hash que se guarda y se compara.
pesos(valor)	Formatea un double como moneda en formato colombiano (\$ 280.000.000).
aEntero(valor, defecto) / aDoble(valor, defecto)	Parsing seguro de parámetros de formulario, sin lanzar excepción si el dato es inválido.
esc(texto)	Escapa caracteres HTML especiales para prevenir XSS.
colorEstado(estado)	Traduce un estado de negocio (APROBADA , PENDIENTE , RECHAZADA , etc.) a una clase de color Bootstrap (success , warning , danger , secondary).

4.3 seguridad.jspf — autorización por rol (ver §3.2)

4.4 cabecera.jspf — layout superior

Emite el `<!DOCTYPE html>` , metadatos, fuentes de Google Fonts, Bootstrap 5 y Bootstrap Icons vía CDN, el CSS propio, y la barra de navegación (`navbar`), que se arma **condicionalmente** según `rolSesion` :

- Todos ven el enlace a **Propiedades**.
- Un `CLIENTE` ve además **Mi espacio**; una `INMOBILIARIA` ve **Gestión inmobiliaria**; un `ADMINISTRADOR` ve **Administración**.
- Si hay sesión, se muestra el nombre, un badge con el rol y el botón **Salir**; si no, el botón **Ingresar**.
- También imprime alertas Bootstrap si la URL trae `?msg=` (éxito) o `?err=` (error), patrón usado en casi todas las redirecciones tras una operación de escritura.

4.5 pie.jspf — layout inferior

Cierra el `<main>` , imprime un pie de página simple y carga el bundle JS de Bootstrap.

5. Páginas públicas

Página	Rol requerido	Función
index.jsp	Ninguno	Portada, buscador rápido, tarjetas informativas.
propiedades.jsp	Ninguno	Catálogo con filtros por texto libre, ciudad y tipo de propiedad; consulta con <code>LEFT JOIN</code> a <code>imagen_propiedad</code> y agrupación por propiedad.
detalle.jsp	Ninguno (acciones visibles según rol)	Ficha completa de una propiedad: datos, características (N:M), galería de imágenes; muestra botones de <i>Agendar visita</i> / <i>Guardar favorito</i> solo si el usuario en sesión es <code>CLIENTE</code> .
login.jsp / acceso.jsp	Ninguno	Formulario e in-validación de inicio de sesión (ver §2 y §3.3).
registro.jsp / registro_guardar.jsp	Ninguno	Alta de cuenta nueva; siempre se asigna el rol <code>CLIENTE</code> por defecto; valida formato de correo, longitud de clave y confirmación, y captura <code>SQLIntegrityConstraintViolationException</code> / código de error <code>1062</code> para detectar correo duplicado.
logout.jsp	Sesión activa	Invalida la sesión (<code>session.invalidate()</code>) y redirige al inicio.
acceso-denegado.jsp	—	Página mostrada cuando el filtro o <code>seguridad.jspf</code> rechazan el acceso a una ruta por rol incorrecto.

Detalle de propiedades.jsp : la consulta principal usa parámetros opcionales combinados con `OR (?=' ' OR column = ?)` , patrón que permite reutilizar el mismo `PreparedStatement` con o sin cada filtro (texto, ciudad, tipo) sin construir SQL dinámico por concatenación.

6. Módulo Cliente (/cliente/*)

Rol exigido: `CLIENTE` . Permite buscar, guardar y gestionar la relación del cliente con las propiedades.

Página	Función
panel.jsp	Panel de bienvenida con contadores (favoritos, citas pendientes, solicitudes en revisión) mediante una consulta con tres subconsultas escalares.
favoritos.jsp / favorito.jsp	Lista de propiedades guardadas / inserción de un nuevo favorito (<code>INSERT INTO</code>

	favorito); captura violación de clave única si ya existía.
agendar.jsp / guardar_cita.jsp	Formulario y guardado de una cita (INSERT INTO cita); la tabla tiene una restricción UNIQUE (id_propiedad, fecha_hora) que evita duplicar horario, capturada como error amigable.
citas.jsp	Historial de citas del cliente con su estado (coloreado por colorEstado).
nueva_solicitud.jsp / guardar_solicitud.jsp	Formulario para radicar una solicitud de compra o arriendo sobre una propiedad disponible; al guardar, inserta en solicitud y en documento_solicitud dentro de una misma transacción.
solicitudes.jsp	Lista de solicitudes del cliente con su documento asociado y estado.
perfil.jsp / guardar_perfil.jsp	Consulta y actualización de nombre, teléfono y dirección; usa INSERT ... ON DUPLICATE KEY UPDATE sobre perfil porque la relación usuario - perfil es 1:1 y puede no existir todavía.

7. Módulo Inmobiliaria (/inmobiliaria/*)

Rol exigido: INMOBILIARIA . Todas las consultas filtran siempre por i.id_usuario = idUsuarioSesion (uniendo propiedad → inmobiliaria), de modo que cada agente solo ve y modifica **sus propias** propiedades, citas y solicitudes.

Página	Función
panel.jsp	Resumen con contadores propios: propiedades activas, citas por atender, solicitudes en revisión.
propiedades.jsp	Listado de propiedades del agente con acciones Editar y Dar de baja .
guardar_propiedad.jsp	Formulario de alta/edición; si recibe id , precarga los datos existentes (validando que la propiedad pertenezca al agente en sesión).
guardar_propiedad_accion.jsp	Procesa el formulario anterior: si id==0 inserta una propiedad nueva ligada a la inmobiliaria del usuario; si no, actualiza; además inserta la imagen principal (si se indicó URL) y las características seleccionadas (relación N:M propiedad_caracteristica), todo en una transacción con commit / rollback.
baja_propiedad.jsp	Baja lógica: UPDATE propiedad SET activo=0, estado='INACTIVA' (no se elimina el registro).
citas.jsp / actualizar_cita.jsp	Lista de citas recibidas sobre las propiedades del agente, con botones para Aprobar (CONFIRMADA) o Rechazar (CANCELADA).
solicitudes.jsp / actualizar_solicitud.jsp	Lista de solicitudes recibidas con botones para Aprobar / Rechazar .
reportes.jsp	Cuatro reportes SQL propios del agente: propiedades con ciudad/tipo/inmobiliaria, características por propiedad (GROUP_CONCAT sobre la relación N:M), propiedades sin citas (LEFT JOIN + HAVING COUNT(...)=0) y citas agrupadas por estado (GROUP BY + HAVING).

8. Módulo Administrador (/administrador/*)

Rol exigido: ADMINISTRADOR . Tiene alcance global (no se filtra por propietario).

Página	Función
panel.jsp	Accesos directos a Usuarios, Catálogos, Auditoría y Reportes.
usuarios.jsp / actualizar_usuario.jsp	Lista todos los usuarios con su(s) rol(es) (GROUP_CONCAT) y estado; permite cambiar el rol (borra e inserta en usuario_rol) o activar/inactivar la cuenta (UPDATE usuario SET activo = NOT activo), en transacción.
catalogos.jsp / guardar_catalogo.jsp	Formularios para agregar ciudades, tipos de propiedad y características; un único JSP de guardado decide la tabla destino según el parámetro tipo.
auditoria.jsp	Consulta cronológica de la tabla auditoria con el nombre del usuario responsable (LEFT JOIN , por si el registro no tiene usuario asociado).
reportes.jsp	Reportes globales: propiedades por ciudad y solicitudes por estado, ambos con GROUP BY / HAVING COUNT(...)>0.

9. Base de datos — inmobiliaria_uts

Script: sql/01_modelo_inmobiliaria_mariadb.sql . Crea la base en utf8mb4 , **15 tablas** normalizadas hasta 3FN, sus relaciones (1:1, 1:N y N:M) y datos de prueba (roles, ciudades, tipos, características, 10 usuarios, propiedades, citas, solicitudes, favoritos y auditoría).

9.1 Tablas y su propósito

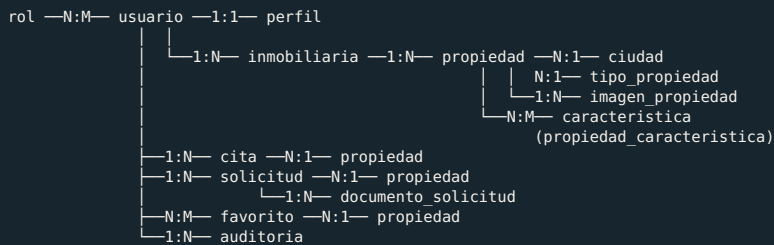
Tabla	Tipo de relación relevante	Descripción
rol	—	Catálogo de roles (CLIENTE , INMOBILIARIA , ADMINISTRADOR).
usuario	—	Cuenta de cualquier persona (nombre, correo único,

		teléfono, clave cifrada, estado activo).
usuario_rol	N:M entre usuario y rol	Permite, en el modelo, que un usuario tenga varios roles (aunque el alta pública siempre asigna solo CLIENTE).
perfil	1:1 con usuario	Datos adicionales del usuario (documento, dirección, ciudad); id_usuario es UNIQUE.
inmobiliaria	1:N desde usuario	Empresa/agente inmobiliario asociado a un usuario con rol INMOBILIARIA; NIT único.
ciudad	—	Catálogo de ciudades y departamento.
tipo_propiedad	—	Catálogo de tipos (Apartamento, Casa, Local, Oficina, Lote, Finca, Bodega, Penthouse, Apartaestudio, Consultorio).
propiedad	1:N desde inmobiliaria, ciudad, tipo_propiedad	Inmueble publicado: matrícula única, título, descripción, dirección, precio, área, habitaciones, baños, estado (DISPONIBLE / INACTIVA /...), baja lógica con activo.
imagen_propiedad	1:N desde propiedad	URLs de imágenes con texto alternativo y orden de visualización.
caracteristica	—	Catálogo de amenidades (Parqueadero, Piscina, Balcón, etc.).
propiedad_caracteristica	N:M entre propiedad y caracteristica	Tabla puente con clave primaria compuesta.
cita	1:N desde propiedad y usuario	Visita agendada; UNIQUE (id_propiedad, fecha_hora) evita doble reserva del mismo horario; estado PENDIENTE / CONFIRMADA / CANCELADA.
solicitud	1:N desde propiedad y usuario	Trámite de COMPRA o ARRIENDO; estado EN_REVISION / APROBADA / RECHAZADA.
documento_solicitud	1:N desde solicitud	Documento(s) adjuntos a una solicitud, con su propio estado.
favorito	N:M entre usuario y propiedad	Clave primaria compuesta (id_usuario, id_propiedad).
auditoria	1:N desde usuario (opcional)	Bitácora de acciones (CREAR, ACTUALIZAR, CONSULTAR) sobre cualquier tabla, con id_usuario nulo permitido para eventos del sistema.

9.2 Reglas de integridad destacadas

- Todas las relaciones están implementadas con FOREIGN KEY, con ON UPDATE CASCADE en todos los casos y ON DELETE ajustado al negocio: CASCADE en tablas puente/dependientes (usuario_rol, propiedad_caracteristica, favorito, imagen_propiedad) y RESTRICT donde borrar el padre no debería ser posible mientras existan hijos (propiedad, cita, solicitud, perfil, inmobiliaria, auditoria).
- Restricciones UNIQUE: usuario.correo, inmobiliaria.nit, propiedad.matricula_inmobiliaria, rol.nombre, ciudad.nombre, tipo_propiedad.nombre, caracteristica.nombre, y la compuesta cita(id_propiedad, fecha_hora).
- Baja lógica (no borrado físico) mediante columnas activo/activa en usuario, propiedad, ciudad, tipo_propiedad, caracteristica, inmobiliaria.
- Valores por defecto de negocio: usuario.activo=1, propiedad.estado='DISPONIBLE', cita.estado='PENDIENTE', solicitud.estado='EN_REVISION', documento_solicitud.estado='PENDIENTE'.

9.3 Diagrama entidad-relación (simplificado)



9.4 Extracto del script (DDL de las tablas centrales)

```

CREATE TABLE usuario (
  id_usuario INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(80) NOT NULL,
  correo VARCHAR(120) NOT NULL UNIQUE,
  telefono VARCHAR(30),
  clave VARCHAR(64) NOT NULL,
  activo TINYINT(1) NOT NULL DEFAULT 1,
  creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE propiedad (
  id_propiedad INT AUTO_INCREMENT PRIMARY KEY,

```

```

id_inmobiliaria INT NOT NULL,
id_ciudad INT NOT NULL,
id_tipo INT NOT NULL,
matricula_inmobiliaria VARCHAR(40) NOT NULL UNIQUE,
titulo VARCHAR(150) NOT NULL,
descripcion TEXT,
direccion VARCHAR(180) NOT NULL,
precio DECIMAL(14,2) NOT NULL,
area DECIMAL(10,2) NOT NULL,
habitaciones INT NOT NULL DEFAULT 0,
banos INT NOT NULL DEFAULT 0,
estado VARCHAR(20) NOT NULL DEFAULT 'DISPONIBLE',
activo TINYINT(1) NOT NULL DEFAULT 1,
creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (id_inmobiliaria) REFERENCES inmobiliaria(id_inmobiliaria)
    ON DELETE RESTRICT ON UPDATE CASCADE,
FOREIGN KEY (id_ciudad) REFERENCES ciudad(id_ciudad)
    ON DELETE RESTRICT ON UPDATE CASCADE,
FOREIGN KEY (id_tipo) REFERENCES tipo_propiedad(id_tipo)
    ON DELETE RESTRICT ON UPDATE CASCADE
);

CREATE TABLE cita (
    id_cita INT AUTO_INCREMENT PRIMARY KEY,
    id_propiedad INT NOT NULL,
    id_usuario INT NOT NULL,
    fecha_hora DATETIME NOT NULL,
    estado VARCHAR(20) NOT NULL DEFAULT 'PENDIENTE',
    observaciones VARCHAR(300),
    UNIQUE KEY uq_cita_horario (id_propiedad, fecha_hora),
    FOREIGN KEY (id_propiedad) REFERENCES propiedad(id_propiedad)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

```

(El script completo crea las 15 tablas, sus llaves foráneas y ~90 filas de datos de prueba; ver `sql/01_modelo_inmobiliaria_mariadb.sql` en el repositorio.)

9.5 Datos de prueba y usuarios

El script inserta 10 usuarios, todos con contraseña `Clave123` (cifrada con `SHA2(correo:Clave123, 256)`):

Rol	Correo(s) de ejemplo
Cliente	ana@ejemplo.com, luis@ejemplo.com, marta@ejemplo.com, diego@ejemplo.com, sofia@ejemplo.com
Inmobiliaria	agente1@ejemplo.com, agente2@ejemplo.com, agente3@ejemplo.com
Administrador	admin@ejemplo.com
Visitante demo	visitante@ejemplo.com

También se cargan 10 ciudades (Santander y otros departamentos), 10 tipos de propiedad, 10 características, 10 inmobiliarias, 10 propiedades con imagen y características asociadas, 10 citas, 10 solicitudes con su documento y 10 registros de auditoría.

10. Configuración y despliegue

10.1 Conexión a la base de datos

Definida en `web/WEB-INF/jspf/conexion.jspf`:

```

URL:      jdbc:mariadb://localhost:3306/inmobiliaria_uts?useUnicode=true&characterEncoding=UTF-8
Usuario:  bryan
Clave:    root   (marcado en el código como TODO a reemplazar antes de un despliegue real)

```

10.2 Pasos de instalación

1. **Crear la base de datos:** `mysql -u root -p < sql/01_modelo_inmobiliaria_mariadb.sql`
2. **Compilar el filtro** contra el Servlet API de Tomcat y dejar el `.class` en `web/WEB-INF/classes/com/uts/inmobiliaria/`.
3. **Desplegar** copiando la carpeta `web/` a `TOMCAT/webapps/parcialEntrega` y reiniciar Tomcat.
4. Acceder a `http://localhost:8080/parcialEntrega/`.

10.3 `web.xml` — configuración de la aplicación

- Página de bienvenida: `index.jsp`.
- Registro del `FiltroControlAcceso` sobre las tres rutas de rol.

- `session-timeout` de 30 minutos.
- Codificación forzada a `UTF-8` para todos los `*.jsp` mediante `jsp-property-group`, con `trim-directive-whitespaces` activado para reducir espacios en blanco en el HTML generado.

11. Resumen de cobertura funcional

- **CRUD completo de propiedades** (inmobiliaria), con relación N:M a características e imágenes 1:N.
- **Gestión de citas** con validación de horario único y flujo de aprobación/rechazo.
- **Gestión de solicitudes** de compra/arriendo con documento adjunto y flujo de aprobación/rechazo.
- **Favoritos** del cliente (N:M usuario-propiedad).
- **Perfil** de usuario (relación 1:1 con `perfil`).
- **Administración** de usuarios/roles, catálogos (ciudad, tipo, característica) y auditoría.
- **Reportes** con `JOIN`, `LEFT JOIN`, `GROUP BY / HAVING` y `GROUP_CONCAT`, tanto a nivel de inmobiliaria como globales.
- **Seguridad en dos capas** (Filter + fragmento JSP), contraseñas cifradas, consultas parametrizadas y escape de salida HTML.

Este documento describe la versión del proyecto contenida en el archivo `parcialEntrega.zip` entregado.